
Reinforcement Learning for Autonomous Navigation in a 2D Environment.

Eirik Varnes
Università di Bologna
`eirik.varnes@studio.unibo.it`

Abstract

Autonomous navigation is a key component of an autonomous underwater vehicle (AUV). In this project, we explore reinforcement learning (RL) as an alternative to traditional rule-based algorithms. The project focuses on the incremental development of both the environment and learning agent to gain a better understanding of how such systems can be designed.

1 Introduction

Autonomous navigation, especially for underwater vehicles that must work in unmapped areas or environments that change, is very challenging for the usual control methods. Reinforcement Learning offers a strong framework for an agent to learn good navigation strategies by interacting with its surroundings, making it a fitting choice for these tasks.

This work follows an iterative development approach. It begins with basic tests in a custom 2D grid world, where complexity in the environment (for example, obstacles) and the agent's perception (for example, LiDAR-like sensors and memory buffers) were added step by step. During this phase, we evaluated both tabular Q-learning and Deep Q-Networks (DQNs) under conditions with limited observability.

In later stages, the project moved on to a more advanced 2D simulation for an AUV. In this phase, we introduced a continuous space with physics-based dynamics and continuous control. To accommodate those changes, we adjusted the reward function and implemented the TD3 actor-critic algorithm to handle continuous action spaces.

This report describes the entire development path, presenting the methodology, experimental results, and discussion for each stage, and concludes with overall findings.

2 Foundational Approach

The scope of this project is mainly set by the reinforcement learning (RL) concepts and principles from the Autonomous and Adaptive Systems course. The agent's neural network implementations were built using the PyTorch framework.

The custom environments, including the initial grid world and the later Autonomous Underwater Vehicle (AUV) simulation, were created to follow the standards and interface of the OpenAI Gym (now Gymnasium) framework to ensure normal agent-environment interactions.

The thematic parts of this project, especially those about AUV navigation, sensor modeling (like LiDAR and sonar), and environmental physics, are based on knowledge from my personal interest in these fields.

3 Method

The project methodology went through several development stages, aiming to explore and apply reinforcement learning (RL) methods for autonomous navigation tasks. An important consideration throughout was the use of partial observations, reflecting realistic conditions where an agent—such as an autonomous underwater vehicle (AUV), must operate without full prior knowledge of its surroundings. This section describes these stages, starting with foundational work in a custom grid world environment and then moving on to a more complex AUV simulation.

3.1 Early Explorations, Algorithm Transition, and Obstacle Challenges

A new environment, AUVEnv, was developed to operate in a continuous 2D workspace defined by `grid_size` (e.g., 200×200 units) and `resolution` (e.g., 0.1 units per internal grid cell). This setup allowed the AUV’s position and orientation to be represented with floating-point precision. Obstacles were stored in an occupancy grid, with layouts either loaded from a static map via `build_maps` or generated randomly using a cellular automaton through `build_random_maps`. At the start of each episode, the AUV and its goal dock(s) were spawned at random, obstacle-free locations.

A key extension in this phase was the introduction of physics-based movement, enabled by a `use_physics` flag. When physics are enabled, motion is no longer purely kinematic. Instead, the AUV’s linear velocity and position evolve according to applied thrust and drag forces, and its heading is adjusted by applied torque. The net force combines thrust (aligned with the AUV’s heading) and linear drag (proportional to velocity), creating acceleration that is integrated over time. In this way, enabling `use_physics` adds inertia and realistic dynamics, making control significantly more challenging.

3.2 Enhanced Perception with LiDAR and Memory with History Buffer

To fix the problem where the agent could not see obstacles, its observations were expanded with LiDAR-like sensors. These sensors gave the DQN four distance measurements to the closest obstacle or boundary in each cardinal direction, up to a set maximum range. By adding this information, the agent gained clear local awareness of its surroundings. Implementing tabular Q-learning became obviously impractical for this larger state space, which confirmed the need for the DQN’s ability to approximate functions. With LiDAR input, the DQN’s success rate rose to about 75.8%, showing that it could learn to avoid obstacles. Still, in more complex maze-like layouts, the agent sometimes got stuck on sub-optimal routes or local traps.

To handle those remaining issues, especially in maze scenarios where a single observation is not enough, a short-term memory feature was added using a history buffer. The DQN’s state was extended to include a series of recent observations (both the relative position and LiDAR readings). The idea was that having temporal context would help the agent distinguish between similar-looking states and detect patterns like oscillations or dead ends previously encountered. This change raised the success rate further to approximately 93%, with most failures occurring only when the goal was physically unreachable.

3.3 AUV Environment Design and Dynamics

A new environment, AUVE_{env}, was built with a continuous 2D operating area determined by `grid_size` (for example, 200×200 units) and `resolution` (for example, 0.1 units per internal grid cell for obstacle representation). This made it possible to have floating-point values for both the position and orientation of the AUV. Obstacles were represented by an occupancy grid, with options for layouts that are either statically defined (using `build_maps`) or generated randomly via a cellular automaton method (using `build_random_maps`). In each episode, the AUV and its goal dock(s) were placed at random coordinates in areas of the map free from obstacles.

A major aspect of this stage was adding the option for physics-based movement, controlled by the `use_physics` flag. When `use_physics` is true, motion is no longer purely kinematic. Instead, the AUV’s linear velocity (v) and position (p) update according to forces, and its heading (θ) updates according to torque. The net force acting on the AUV is calculated from thrust and drag:

$$F_{net} = F_{thrust} + F_{drag}.$$

Thrust acts along the AUV’s current heading. Drag is modeled as a linear force:

$$F_{drag} = -c_d v,$$

where c_d is the `drag_coef`. Acceleration is then

$$a = \frac{F_{net}}{m},$$

where m is the mass. The state updates use Euler integration with a time step Δt (for example, 0.1 seconds, set by `dt`):

$$\begin{aligned} v_{t+1} &= v_t + a_t \Delta t, \\ p_{t+1} &= p_t + v_{t+1} \Delta t, \\ \theta_{t+1} &= \theta_t + \left(\frac{\text{torque}_t}{m} \right) \Delta t. \end{aligned}$$

The environment can also include configurable ocean currents, which add another velocity component to the AUV’s motion. This physics model introduces inertia and makes the control task much more challenging.

3.4 AUV Action and Observation Spaces

The AUVE_{env} supports two modes of control, selectable via the `use_discrete_actions` flag. In discrete mode, the agent chooses from a small set of predefined maneuvers that combine fixed thrust and turning rates (e.g., moving forward, turning in place, or moving forward while turning). This simplifies control by limiting choices to a handful of representative actions. In continuous mode, the agent outputs a two-dimensional action vector consisting of a thrust command and a torque command. These outputs are produced by the neural network in a normalized range and then scaled to the vehicle’s allowable thrust and turning limits. Continuous actions allow the agent to select precise levels of thrust and rotation, enabling smoother and more flexible control.

Under partial observability, each observation is built from three key elements. First, a 360-degree sonar sensor provides multiple normalized distance readings around the AUV along with a binary mask indicating which beams detect obstacles or boundaries nearby. This gives the agent local proximity information. Second, goal-related information is included: for each target dock, the observation contains the (normalized) distance to that dock and the sine and cosine of the bearing angle relative to the AUV’s heading. This encoding allows the agent to infer both how far it is from each goal and in which direction it must move. Third, when `use_history` is activated, the agent’s recent action commands are recorded in a history buffer and included in the observation vector. This simple replay of past actions serves as a short-term memory, allowing the agent to distinguish between states that appear identical in a single sensor snapshot. At each time step, the final observation consists of the sonar distance readings and hit mask, the encoded goal information (distance and bearing), and, if enabled, the sequence of recent actions. These elements are concatenated into one vector, which is then fed into the policy or value network.

3.5 AUV Reward Function Design

The sparse reward scheme from the grid world (receiving a positive reward only upon reaching the goal and a small negative reward per time step) was inadequate for the larger, more complex AUVEnv, since finding the goal by random exploration alone was extremely unlikely. Consequently, a shaped reward function was introduced to give the agent denser feedback. Determining the right combination of reward components and their relative weights required extensive experimentation, including examining typical reward magnitudes to keep everything balanced.

The final shaped reward function comprised several parts. First, a large positive docking reward was used to strongly encourage successfully reaching the target. Second, a small negative cost per time step incentivized the agent to find shorter paths. Third, a penalty was applied when the AUV got too close to obstacles or walls whenever its minimum sonar reading fell below a predefined safety threshold, discouraging the agent from hugging boundaries. Fourth, a positive progress reward was added, proportional to how much closer the AUV moved toward the goal in a single step, encouraging steady forward movement. Fifth, a penalty proportional to the amount of turning effort discouraged excessive or unnecessary torque.

For continuous action mode, analogous coefficients were selected so that each reward component remained balanced relative to the scale of continuous control signals.

3.6 Agent Implementations and Rationale

An initial Deep Q-Network (DQN) agent, using a Multi-Layer Perceptron (MLP) for function approximation along with experience replay and target networks, was first applied to AUVEnv. This agent used discrete actions and the shaped reward, showing reasonable performance when physics was disabled. However, discrete actions simplify continuous AUV control. To enable finer control, the environment was switched to a continuous action space, allowing direct thrust and torque selection. Since standard DQNs cannot handle continuous actions, the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm was implemented. TD3 is an actor-critic method with an actor network producing continuous actions and twin critic networks evaluating Q-values. It reduces overestimation by using the minimum of two critics, updates the policy and target networks less frequently, and applies target policy smoothing by adding noise to target actions. These features enhance stability in continuous control. TD3 was first validated in the no-physics AUV setup with continuous actions, where it frequently outperformed DQN. Its demonstrated effectiveness and design for stable continuous control motivated its use in the physics-enabled environment.

4 Experiments

4.1 Initial Agent Performance and Obstacle Impact

In the simple 5×5 grid world without obstacles, the tabular Q-learning agent learned rather well; after some exploration, its total reward per episode quickly climbed and stayed near zero, which shows it reached the goal efficiently. For the larger 20×20 grid without obstacles, the Q-learning agent also managed to learn: its 1 000-step moving average of rewards settled around −10, achieving a 98.1 % success rate. However, it took quite long—around 20 000 to 30 000 episodes—for this convergence to happen.

By contrast, a DQN agent trained on the same 20×20 grid without obstacles reached 100 % success much faster. Its 10-step moving average of rewards moved close to zero with relatively small variance in under 1 000 episodes, which clearly shows how DQN scales better to larger state spaces—even in this simpler setting.

When obstacles were introduced into the 20×20 grid world, a DQN agent that only knew its relative position to the goal (no LiDAR sensors) still managed a 59.7 % success rate in test runs. Learning was nevertheless very challenging: the total reward per episode fluctuated a lot, and even though a 20-step moving average showed some improvement, it stayed low (around −25 to −50) and was quite unstable. This demonstrated that the agent struggled to learn reliable obstacle avoidance without direct sensory input or a way to understand why some actions did not change its state as expected.

4.2 Results with Enhanced Perception and Memory

When we added four-directional LiDAR sensors, the DQN agent suddenly could see local obstacles around itself, and performance in the 20×20 grid with obstacles improved noticeably. The success rate climbed to 82.3 %, and when it did reach the goal, it took on average 12.8 steps. Looking at the learning curve, the 10-step moving average of total rewards rose more steadily and stayed higher than before, which shows the agent was effectively using the LiDAR measurements. The slightly higher average step count (12.8 vs. 12.3) suggests the agent was choosing more deliberate routes to avoid obstacles, even if they were a bit longer.

Next, we gave the DQN agent a short history buffer that stored recent observations (including LiDAR data). This further boosted performance in the difficult 20×20 grid with obstacles: the success rate went up to 94.3 %. The learning curve’s 10-step moving average of rewards was generally higher and more stable than with LiDAR alone. Successful runs now took about 18.4 steps on average. This increase indicates the agent with memory could handle more complex, maze-like layouts requiring longer detours—situations where an agent without memory might have gotten stuck or bounced between walls. The history buffer probably helped the agent recognize when it was repeating ineffective moves. Failures at this stage mostly happened when the goal was completely hidden behind obstacles.

4.3 Performance in Kinematic AUV Environment (DQN and TD3)

We first tested a DQN agent using discrete actions and the shaped reward in the AUV environment without physics. In a static map, this DQN reached a 97.6 % success rate and took on average 29.9 steps when successful. Its learning curve, measured as a 50-step moving average of total rewards, stayed close to zero most of the time—showing that the agent learned to reach the goal without collecting many penalties. In randomly generated maps, the DQN’s success rate was 86.0 %, averaging 34.3 steps for successful runs. The learning curve in this case did show improvement, with the 50-step moving average of rewards rising into positive values, but it was more variable than in the static case, reflecting the extra challenge of generalising across different layouts. When tested on a completely new, randomly generated map not seen during training, the success rate was still a strong 91.4 %.

After that, we evaluated a TD3 agent, which uses continuous actions, in the AUV environment without physics. TD3 generally outperformed the DQN. In the static AUV map, TD3 reached a 99.0 % success rate, with successful episodes taking on average 36.1 steps. Its learning curve showed very rapid improvement: the 50-step moving average of total rewards climbed quickly to high values, indicating efficient policy learning. In randomly generated maps without physics, TD3 also did very well, with a 91.0 % success rate and an average of 33.1 steps for successful runs. The learning curve again showed strong, stable rewards. These results suggest that an agent capable of continuous control can learn more effective navigation policies in a purely kinematic AUV environment.

4.4 Impact of Physics on TD3 Performance

When physics-based dynamics were enabled, the TD3 agent faced major difficulties. In a static AUV environment with physics turned on, the success rate dropped sharply, and successful runs required many more steps on average. The learning curve remained very low throughout training, showing that the agent collected substantial penalties and struggled to reach the goal. In randomly generated environments with physics, performance deteriorated even further: successful episodes were extremely rare and usually involved just a single step, suggesting they were trivial cases. The learning curve for these scenarios stayed flat, with only occasional peaks when a rare success occurred. This large drop in performance highlights how much more challenging the task becomes once inertia, drag, and other physics factors are added, challenges that the current TD3 setup and reward design were not able to overcome.

5 Discussion

This project investigated the use of reinforcement learning for autonomous navigation, beginning with a simple grid world and advancing to a more complex 2D AUV environment. The experiments at each phase yielded insights into the strengths and weaknesses of different RL agents and environment configurations.

A recurring theme was the reliance on partial observations. In the initial grid world, a basic state representation, using only the relative position to the goal, proved inadequate once obstacles were introduced but not directly sensed. With a DQN agent, success rates rose from 59.7% when obstacles were unseen to 82.3% after adding LiDAR-like sensors, and then to 94.3% once a history buffer provided short-term memory. These results highlight how essential both sufficient sensory input and some form of memory are for effective navigation in environments where the agent cannot observe everything at once.

The move from tabular Q-learning to Deep Q-Networks in the grid world illustrated the importance of algorithmic scalability. In small grids, Q-learning was effective, but in a 20×20 grid, DQN achieved 100% success in fewer than 1 000 episodes, whereas Q-learning reached 98.1% success only after 20 000–30 000 episodes. This clearly justified using neural network-based methods for larger state spaces. In the AUV environment without physics, the continuous-action TD3 agent generally outperformed the discrete-action DQN, for example, achieving 99.0% versus 97.6% success in static maps and 91.0% versus 86.0% in random maps, suggesting that continuous action outputs better match environments with inherently continuous dynamics.

Introducing physics-based dynamics in the AUV environment proved especially challenging. The TD3 agent’s success rate dropped from 99.0% without physics to just 25.0% when physics were enabled, and from 91.0% to 0.4% in randomly generated maps. This dramatic decline indicates that factors such as inertia, drag, and more complex interactions require more sophisticated approaches. It also underscored the importance of reward shaping: the simple sparse reward used in the grid world was insufficient here, and a carefully designed, denser reward function was crucial for encouraging useful behaviors.

Despite these insights, the project has limitations. The most notable is the poor TD3 performance once physics were enabled. Although the sensor and physics models added realism, they remained simplifications. Exploration strategies were relatively basic, and further hyperparameter tuning might improve results. In addition, other advanced RL algorithms that could address these challenges were not explored.

Looking forward, future work should focus on enhancing AUV agent performance under physics-based dynamics. This might include investigating newer RL methods, designing more sophisticated network architectures, refining reward engineering, and developing better exploration strategies. Increasing simulation realism—such as modelling water effects, improving sensor fidelity, and adding dynamic ocean currents—and potentially integrating RL with classical control or SLAM techniques could also lead to more robust and efficient navigation solutions.

6 Conclusion

This project demonstrated an iterative approach to developing and applying reinforcement learning agents for autonomous navigation, starting from simple grid worlds and advancing to a more complex 2D AUV simulation. The results highlighted how crucial adequate sensory data and memory are in partially observable settings: DQN success rates in grid worlds rose from 59.7% to 94.3% once sensors and a history buffer were added. The scalability advantage of neural network-based agents such as DQN over tabular Q-learning in larger state spaces was also confirmed. In the kinematic AUV simulations, TD3’s continuous control capabilities outperformed the discrete-action DQN (99.0% versus 97.6% success in static environments without physics). However, introducing physics-based dynamics in the AUV environment posed a major challenge: TD3 performance fell sharply (e.g., to 25.0% in static maps and 0.4% in random maps with physics), revealing a key limitation. While carefully shaped rewards were essential for training AUV agents, achieving robust navigation in complex, dynamic systems remains an open avenue for future work, including exploring more advanced agent architectures and algorithms.